Building UAV Control Pipelines using **MAVProxy** and **PyMAVLink**

Agenda

Introduction

- What is MAVLink, MAVProxy, and PyMAVLink
- How they fit into the ArduPilot ecosystem

Part 1 — MAVProxy

Concepts

- MAVProxy overview and role in GCS stack
- Installation and setup
- Connecting to SITL
- Modules, commands, and outputs

Hands-on

- Connect SITL to MAVProxy
- Monitor parameters, heartbeats, and mode changes
- Control vehicle via console

Part 2 — PyMAVLink

Concepts

PYMAVLink overview & architecture

- Connecting to MAVLink endpoints
- Common message types & commands

Hands-on

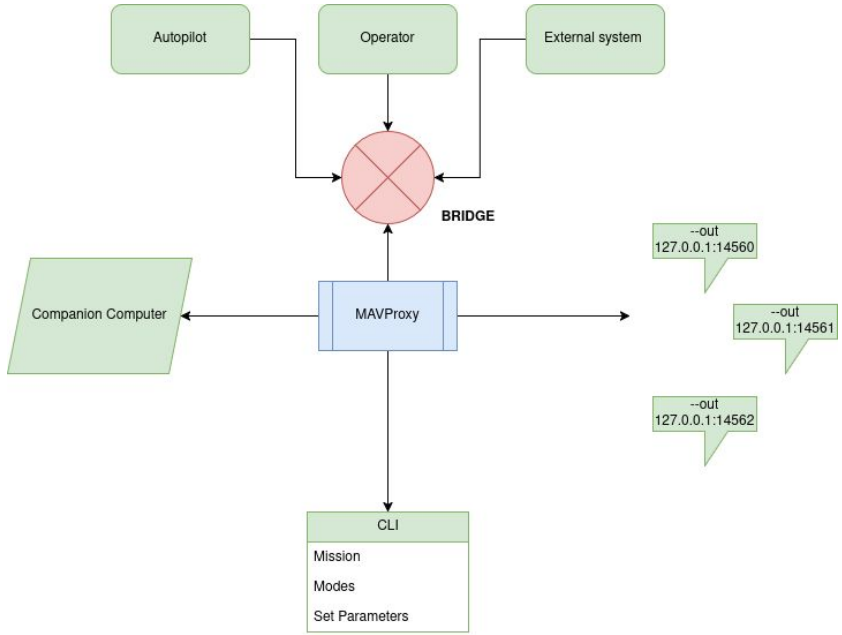
- Write Python scripts to:
    - Arm & takeoff
    - Change flight modes
    - Vehicle connections
    - Arm and Disarm
    - Get vehicle data
    - Send vehicle data

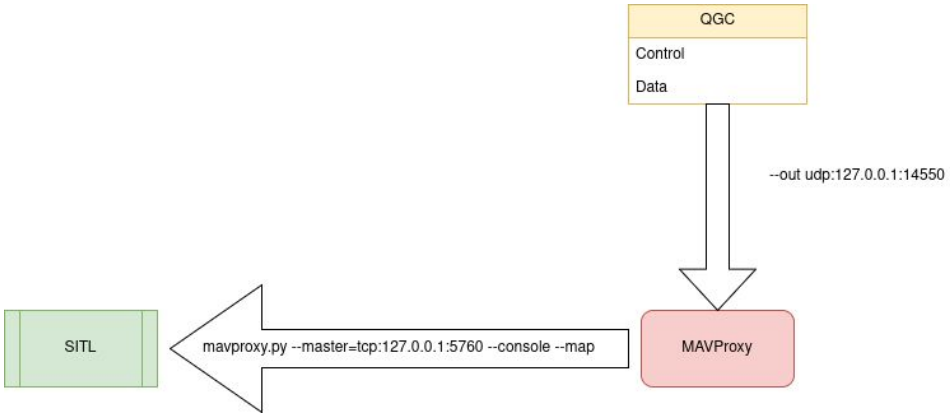
MAVPROXY

Introduction to MAVProxy — The Command-Line Ground Control Station

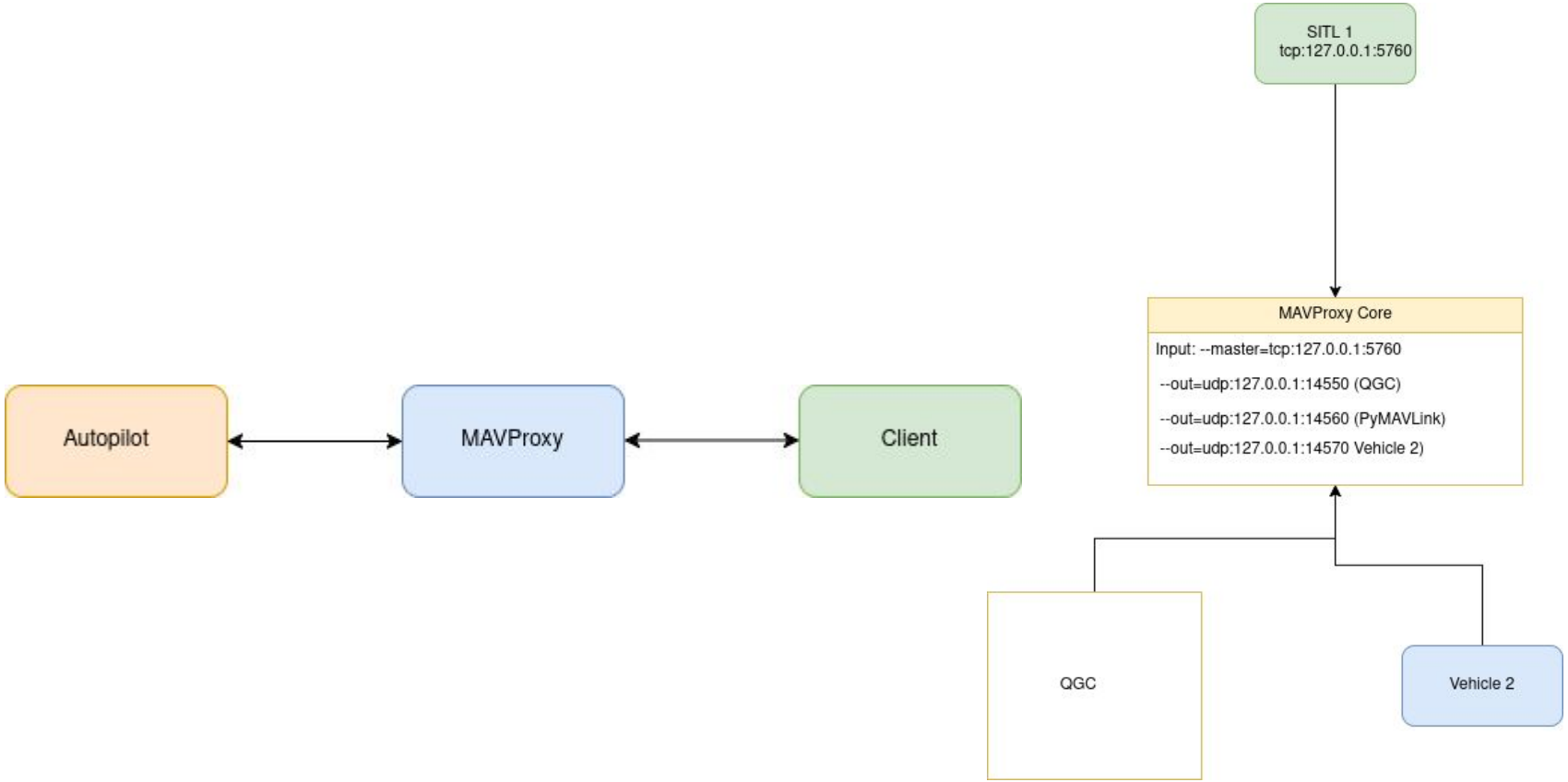
🧠 **What is MAVProxy?**

- A **lightweight, command-line Ground Control Station (GCS)** for MAVLink-enabled vehicles.
- Acts as a **router and monitor** for MAVLink messages between autopilot, SITL, and other GCS tools (like QGroundControl).
- Written in **Python**





MAVProxy as a Communication Hub



Connect SITL Instance + MAVProxy + QGC

step 1

cd && cd ardu-sim/

step 2

./arducopter -S --model + --speedup 1 --defaults parameters/copter.parm -I0

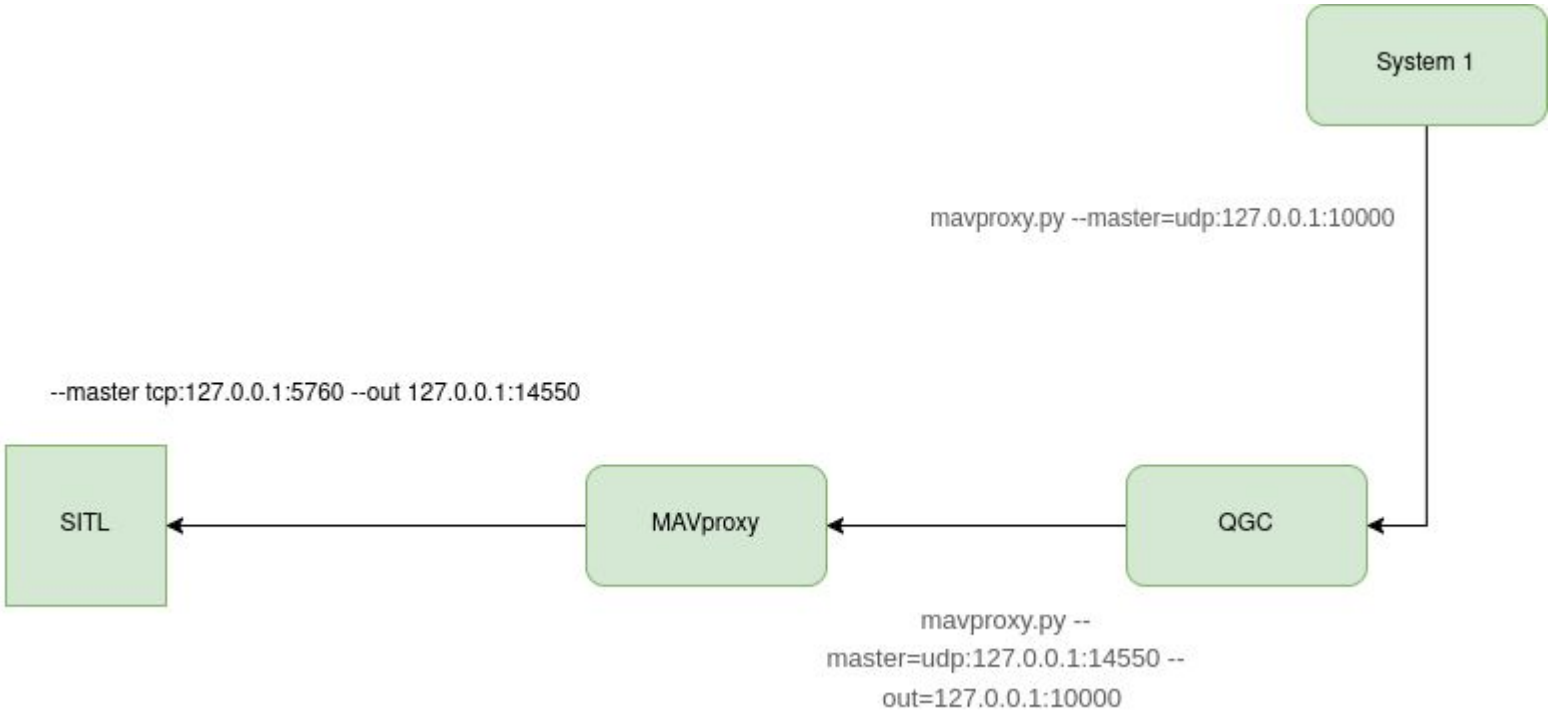
step 3

mavproxy.py --master tcp:127.0.0.1:5760

step 4 Connect to QGC

mavproxy.py --master tcp:127.0.0.1:5760 --out udp:127.0.0.1:14550

Telemetry Forwarding



Try Telemetry Forwarding

Step 1

cd ardu-sim

./ardu-sim.sh

screen -ls

Step 2 - New terminal

mavproxy.py --master=udp:127.0.0.1:14550 --out=127.0.0.1:10000

Step 3 - New terminal (Forward Instance)

mavproxy.py --master=udp:127.0.0.1:10000

Arm Disarm Vehicle using MAVProxy

1. Connect to the vehicle using `**mavproxy.py --master=127.0.0.1:14550 --console --map**`.

2. To arm the vehicle type `**arm throttle**` in command line.

3. To disarm the vehicle type `**disarm**` in command line.

4. If you arm the vehicle and do nothing it automatically disarms after `DISARM_DELAY` seconds.

5. To show disarm delay parameter `param show DISARM_DELAY`.

6. To set disarm delay parameter to 10 seconds `param set DISARM_DELAY 10`.

Change Flight Mode and Takeoff

1. Connect to the vehicle using `**mavproxy.py --master=127.0.0.1:14550 --console --map**`.
2. To change the flight mode `**mode**` in command line.
3. Example `**mode GUIDED**`.
4. Let's take off the vehicle to the 10 meters:
    1. `**mode GUIDED**`
    2. `**arm throttle**`
    3. `**takeoff 10**`
5. Let's get back our vehicle to the ground.
    1. `**mode RTL**` or `**mode LAND**`
6. Let's fly to a location in GUIDED mode.
    1. `**guided -35.36217477 149.16507393 10**`

System command

1. Connect to the vehicle using: `mavproxy.py --master=127.0.0.1:14550 and press tab.`
2. `reboot` command is used to soft restart flight controller.
3. `time` is used to show time on flight controller and computer.
4. `shell` is used to run a shell command.
5. `status` command shows the latest packets.

Long Command

1. Long command enables user to send MAV_CMD_* commands to vehicle.
2. It is used to send COMMAND_LONG messa to the vehicle to give commands during fligh
3. All the parameters are in float.
4. It uses command service of the MAVLink.
5. Usage is: `long COMMAND_NAME PARAM1 PARAM2 PARAM3 PARAM4 PARAM5 PARA PARAM7`
6. Lets takeoff in guided mode using MAV_CMD_NAV_TAKEOFF:
7. arm throttle
    1. `long MAV_CMD_NAV_TAKEOFF 0 0 0 0 0 0 10`
    2. This command needs takeoff altitude in meters as its 7th parameter.

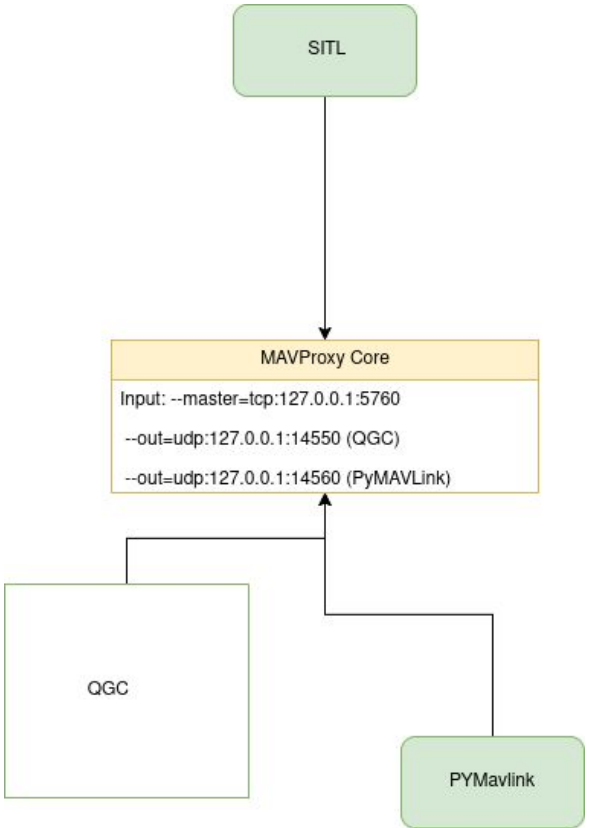
MAV_CMD_NAV_TAKEOFF (22)

Takeoff from ground / hand. Vehicles that support multiple takeoff modes (e.g. VTOL quadplane) should take off using the currently configured mode.

Param (Label)	Description
1 (Pitch)	Minimum pitch (if airspeed sensor present), desired pitch without sensor
2	Empty
3	Empty
4 (Yaw)	Yaw angle (if magnetometer present), ignored without magnetometer. NaN to use the current system yaw heading mode (e.g. yaw towards next waypoint, yaw to home, etc.).
5 (Latitude)	Latitude
6 (Longitude)	Longitude
7 (Altitude)	Altitude

PYMAVLink

Connection with PYMAVLink



Connect to the Vehicle

```
import pymavlink.mavutil

# connect to vehicle

vehicle = pymavlink.mavutil.mavlink_connection(device="udpin:127.0.0.1:14570")


# wait for a heartbeat

vehicle.wait_heartbeat(timeout=5)


# debugging messages

print("Connected to the vehicle")

print("Target system:", vehicle.target_system, "Target component:", vehicle.target_component)
```

Receiving Message from Vehicle

```python
# infinite loop
while True:

    # try to receive a message
    try:

        # receive a message
        message = vehicle.recv_match(type=dialect.MAVLink_system_time_message.msgname, blocking=True)

        # convert received message to dictionary
        message = message.to_dict()

        # for each field name in field names of this message
        for field_name in dialect.MAVLink_system_time_message.fieldnames:

            # if the field is system boot time in milliseconds
            if field_name == "time_boot_ms":
                # print field name and contained field value
                print(field_name, message[field_name])

    # exit on Ctrl+C
    except KeyboardInterrupt:
        print("User interrupt received, exiting.")
        exit(0)

    # bare except to catch all the exceptions
    except Exception as e:

        # print error message
        print(f"Error occurred: {e}")
```

Sending Message to the Vehicle

```
# build up the command message
message = dialect.MAVLink_command_long_message(target_system=vehicle.target_system,
                                               target_component=vehicle.target_component,
                                               command=dialect.MAV_CMD_DO_SEND_BANNER,
                                               confirmation=0,
                                               param1=0,
                                               param2=0,
                                               param3=0,
                                               param4=0,
                                               param5=0,
                                               param6=0,
                                               param7=0)

# send the command to the vehicle
vehicle.mav.send(message)

# do below always
while True:

    # receive a message
    message = vehicle.recv_match(type=[dialect.MAVLink_statustext_message.msgname,
                                       dialect.MAVLink_command_ack_message.msgname],
                                 blocking=True)

    # convert message to dictionary
    message = message.to_dict()

    # show the message
    print(message)
```

Arming and Disarming of Vehicle

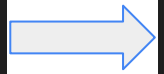
```
# vehicle arm message
vehicle_arm_message = dialect.MAVLink_command_long_message(
    target_system=vehicle.target_system,
    target_component=vehicle.target_component,
    command=dialect.MAV_CMD_COMPONENT_ARM_DISARM,
    confirmation=0,
    param1=VEHICLE_ARM,
    param2=0,
    param3=0,
    param4=0,
    param5=0,
    param6=0,
    param7=0
)

# vehicle disarm message
vehicle_disarm_message = dialect.MAVLink_command_long_message(
    target_system=vehicle.target_system,
    target_component=vehicle.target_component,
    command=dialect.MAV_CMD_COMPONENT_ARM_DISARM,
    confirmation=0,
    param1=VEHICLE_DISARM,
    param2=0,
    param3=0,
    param4=0,
    param5=0,
    param6=0,
    param7=0
)
```

Flight Modes

```
# create change mode message
set_mode_message = dialect.MAVLink_command_long_message(
    target_system=vehicle.target_system,
    target_component=vehicle.target_component,
    command=dialect.MAV_CMD_DO_SET_MODE,
    confirmation=0,
    param1=dialect.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,
    param2=flight_modes[FLIGHT_MODE],
    param3=0,
    param4=0,
    param5=0,
    param6=0,
    param7=0
)
```

Take OFF

```python
# create takeoff command
takeoff_command = dialect.MAVLink_command_long_message(
    target_system=vehicle.target_system,
    target_component=vehicle.target_component,
    command=dialect.MAV_CMD_NAV_TAKEOFF,
    confirmation=0,
    param1=0,
    param2=0,
    param3=0,
    param4=0,
    param5=0,
    param6=0,
    param7=TAKEOFF_ALTITUDE
)
```



```python
# check the pre-arm
while True:

    # catch GLOBAL_POSITION_INT message
    message = vehicle.recv_match(type=dialect.MAVLink_global_position_int_message.msgname,
                                 blocking=True)

    # convert message to dictionary
    message = message.to_dict()

    # get relative altitude
    relative_altitude = message["relative_alt"] * 1e-3

    # print out the message
    print("Relative Altitude", relative_altitude, "meters")

    # check if reached the target altitude
    if TAKEOFF_ALTITUDE - relative_altitude < 1:

        # print out that takeoff is successful
        print("Takeoff to", TAKEOFF_ALTITUDE, "meters is successful")

        # break the loop
        break
```

Land Command

```python
# create land command
land_command = dialect.MAVLink_command_long_message(
    target_system=vehicle.target_system,
    target_component=vehicle.target_component,
    command=dialect.MAV_CMD_NAV_LAND,
    confirmation=0,
    param1=0,
    param2=0,
    param3=0,
    param4=0,
    param5=0,
    param6=0,
    param7=0
)
```

```python
# check the pre-arm
while True:

    # catch GLOBAL_POSITION_INT message
    message = vehicle.recv_match(type=dialect.MAVLink_global_position_int_message.msgname,
                                 blocking=True)

    # convert message to dictionary
    message = message.to_dict()

    # get relative altitude
    relative_altitude = message["relative_alt"] * 1e-3

    # print out the message
    print("Relative Altitude", relative_altitude, "meters")

    # check if reached the target altitude
    if relative_altitude < 1:

        # break the loop
        break

# wait some seconds to land
time.sleep(10)

# print out that land is successful
print("Landed successfully")
```